

Описание функций сглаживания (Python, программистская версия)

1 Выход

1.1 Сигнатура

```
def boxcar(win: int, data: list) -> list
```

1.2 Параметры

Параметр	Тип	Описание
win	int	Размер окна усреднения
data	list[float]	Входной сигнал длиной N

1.3 Алгоритм

```
def boxcar(win, data):
    half = win // 2
    n = len(data)
    avg = []
    for i in range(n):
        start = max(0, i + 1 - half)
        end = min(half + i, n)
        sum_array = sum(data[start:end])
        aver = sum_array / len(data[start:end])
        avg.append(aver)
    return avg
```

Для каждого индекса i вычисляются границы окна $[start, end)$ с обрезкой по краям массива. Вычисляется арифметическое среднее по срезу $data[start:end]$.

1.4 Сложность

Параметр	Значение
Время	$O(N \cdot win)$
Память	$O(N)$
Гранич. условия	Clamping (обрезка окна)

2 Gaussian

2.1 Сигнатура

```
def gaussian(data: list, win: int) -> list
```

2.2 Алгоритм — ядро

```
def gaussian(data, win):
    half = win // 2
    sigma = win / 6.0
    n = len(data)
    k = [math.exp(-0.5*(i - half)**2 / sigma**2)
          for i in range(win)]
    k = [x/sum(k) for x in k]
```

2.3 Алгоритм — дополнение и свёртка

```
p = ([data[i] for i in range(half-1, -1, -1)]
      + data
      + [data[i] for i in range(n-1, n-half-1, -1)])
return [sum(k[j]*p[i+j] for j in range(win))
        for i in range(n)]
```

Ядро Гаусса: $w[k] = \exp(-0.5 \cdot (k - \text{half})^2 / \sigma^2)$, $\sigma = \text{win}/6$. Нормализация: $w_norm[k] = w[k] / \sum w$. Дополнение: симметрическое отражение на `half` элементов. Свёртка: каждый выходной отсчёт — взвешенная сумма по ядру.

2.4 Сложность

Параметр	Значение
Время	$O(N \cdot \text{win})$
Память	$O(N + \text{win})$
Гранич. условия	Mirror reflection (отражение)

3 Progressive

3.1 Сигнатура

```
def progressive(data: list) -> list
```

3.2 Алгоритм

```
def progressive(data):
    result = []
    total = 0.0
```

```

for i, value in enumerate(data):
    total += value
    result.append(total / (i + 1))
return result

```

Накапливается сумма `total`. На каждом шаге `i` вычисляется `total / (i + 1)` — среднее отсчётов от 0 до `i`.

3.3 Сложность

Параметр	Значение
Время	$O(N)$
Память	$O(N)$
Гранич. условия	Нет (окно растёт от 1 до N)

4 Hybrid

4.1 Сигнатура

```

def hybrid(data: list, win: int) -> list

```

4.2 Алгоритм

```

def hybrid(data, win):
    n = len(data)
    box = boxcar(win, data)
    prog = progressive(data)

    out = []
    for i in range(n):
        if i < win:
            out.append(box[i])
        elif i < 2 * win:
            z = (i - win) / win
            k = (1.0 - z) * box[i] + z * prog[i]
            out.append(k)
        else:
            out.append(prog[i])
    return out

```

4.3 Логика

Условие	Действие
$i < \text{win}$	Берётся <code>boxcar[i]</code>
$\text{win} \leq i < 2 \cdot \text{win}$	Линейная интерполяция: $(1-z) \cdot \text{box} + z \cdot \text{prog}$
$i \geq 2 \cdot \text{win}$	Берётся <code>progressive[i]</code>

Коэффициент интерполяции: $z = (i - \text{win}) / \text{win}$, $z \in [0, 1)$.

4.4 Сложность

Параметр	Значение
Время	$O(N \cdot \text{win})$ (из-за boxcar)
Память	$O(N)$
Гранич. условия	Зависит от boxcar (clamping)